ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

ĐỒ HỌA MÁY TÍNH

BÀI TẬP LỚN

Giáo viên hướng dẫn: TS. Lê Thành Sách
Sinh viên: Hồ Trọng Nghĩa - 2252526.

TP. HỒ CHÍ MINH, THÁNG 4/2026

Mục lục

1 Tổng quan chủ đề **2**

2 Mô tả chung **2**

3 Các kỹ thuật và thư viện được sử dụng **2**
3.1 Các thư viện và yêu cầu kỹ thuật . . . 3
3.2 Kiến trúc Entity-Component-System (ECS) . . . 3
3.2.1 Thực thể (Entity) và Thành phần (Component) . . . 3
3.2.2 Hệ thống xử lý (System) . . . 4
3.3 Các hệ thống chuyên biệt . . . 5

4 Các tính năng đã thực hiện **6**
4.1 Bài tập lớn 1: Dựng hình và Trực quan hóa . . . 6
4.2 Bài tập lớn 2: Bộ sinh ảnh tổng hợp . . . 6

5 Hướng dẫn sử dụng **7**
5.1 Điều khiển Camera và Quan sát . . . 7
5.2 Quản lý và Tương tác Vật thể . . . 7
5.3 Công cụ chuyên dụng . . . 7

6 Hình chụp màn hình **8**

7 Kho GitHub & hướng dẫn cài đặt **9**
7.1 Cài đặt . . . 9
7.1.1 Yêu cầu hệ thống . . . 9
7.1.2 Các bước cài đặt & chạy . . . 9
7.1.3 Khắc phục sự cố . . . 9

1 Tổng quan chủ đề

Bài báo cáo này mô tả chi tiết về kiến trúc phần mềm, các thư viện được sử dụng, cũng như các tính năng chính của engine đồ họa 3D được xây dựng nhằm phục vụ cho Bài tập lớn môn Đồ họa Máy tính (Học kỳ II Năm học 2025 - 2026).

Đề tài yêu cầu xây dựng một hệ thống hiển thị (render) các cảnh hình học 2D và 3D sử dụng pipeline đồ họa lập trình được (modern OpenGL với GLSL). Bên cạnh đó, ứng dụng còn tích hợp các tính năng mô phỏng trực quan giải thuật tối ưu hóa (Stochastic Gradient Descent), trực quan hóa cấu trúc phân tử, và đặc biệt là hệ thống sinh ảnh tổng hợp (synthetic image generator) nhằm tạo bộ dữ liệu cho các bài toán thị giác máy tính như phát hiện đối tượng, phân đoạn (segmentation) và ước lượng độ sâu.

2 Mô tả chung

Ứng dụng cung cấp một không gian tương tác 3D toàn diện với các tính năng sau:

- **Quản lý và thao tác đối tượng:** Hỗ trợ tải các mô hình 3D đa định dạng (`.glb`, `.obj`, `.ply`) và khởi tạo các khối hình học cơ bản (Cube, Icosphere, Torus,...). Cho phép người dùng tương tác trực tiếp trong không gian 3D qua hệ thống Gizmo (tịnh tiến, xoay) hoặc qua các thanh trượt trên giao diện.
- **Hệ thống Camera linh hoạt:** Ứng dụng tích hợp hệ thống Arcball Camera hỗ trợ các thao tác xoay (orbit), di chuyển (pan), thu phóng (zoom) và tự động tập trung (focus) vào một đối tượng đang được chọn trong cảnh.
- **Chiếu sáng và Vật liệu:** Sử dụng mô hình chiếu sáng dựa trên vật lý (PBR), cho phép tinh chỉnh các thông số vật liệu như Albedo, Roughness, Metallic. Hệ thống hỗ trợ ánh xạ texture và nhiều loại nguồn sáng khác nhau (Point Light, Directional Light).
- **Trực quan hóa Toán học và Thuật toán:**
  - Dựng lưới bề mặt 3D theo thời gian thực dựa trên các biểu thức toán học do người dùng nhập vào.
  - Mô phỏng quá trình tối ưu hóa của các thuật toán như Stochastic Gradient Descent (SGD) và Momentum trên các hàm mất mát kinh điển (như hàm Rosenbrock), hiển thị quỹ đạo di chuyển của các thuật toán theo thời gian thực.
- **Sinh dữ liệu tổng hợp (Dataset Generation):** Cho phép chụp lại khung hình hiện tại để xuất thành các tập dữ liệu huấn luyện AI. Dữ liệu đầu ra bao gồm ảnh màu RGB, bản đồ độ sâu (depth map) và mặt nạ phân đoạn (segmentation mask).

3 Các kỹ thuật và thư viện được sử dụng

Hệ thống được phát triển hoàn toàn bằng ngôn ngữ Python (yêu cầu phiên bản 3.10 trở lên) kết hợp với các thư viện đồ họa và tính toán hiệu năng cao. Để đảm bảo hiệu suất render, tính ổn định trên đa nền tảng và khả năng mở rộng, ứng dụng áp dụng các kỹ thuật phần mềm tiên tiến và mô hình kiến trúc tách biệt dữ liệu - logic.

3.1 Các thư viện và yêu cầu kỹ thuật

- **OpenGL và quản lý phiên bản**: Hệ thống sử dụng `PyOpenGL` và `PyOpenGL-accelerate` để tương tác với API OpenGL. Ứng dụng tạo context OpenGL 4.5 để tương thích tối đa với các công cụ gỡ lỗi đồ họa chuyên sâu như RenderDoc, mặc dù tập lệnh thực thi vẫn nằm trong giới hạn tính năng của 3.3.

> **Về macOS**
> Hệ thống sẽ yêu cầu OpenGL 4.1 thay vì 4.5 nếu phát hiện macOS. Lớp `Shader` tự động thực hiện patch mã nguồn shader bằng regex tại thời điểm chạy (ví dụ: chuyển `#version 450 core` thành `#version 410 core`).

- **Quản lý cửa sổ và giao diện**: `glfw` được sử dụng để khởi tạo window context và xử lý sự kiện đầu vào. Giao diện người dùng (UI) được xây dựng thông qua `imgui-bundle` (Dear ImGui), cung cấp các bảng điều khiển như Inspector và Scene Panel.
- **Tính toán và xử lý dữ liệu**: `numpy` đóng vai trò trung tâm trong việc thực hiện các phép toán ma trận/vector tốc độ cao, tính toán AABB (Axis-Aligned Bounding Box) trong thời gian thực và biên dịch các hàm toán học để tạo lưới bề mặt 3D.
- **Xử lý tài nguyên**: `trimesh` được sử dụng để phân tách biểu đồ phân cấp (scene graph) từ các định dạng `.glb, .gltf, .obj, .ply`. `Pillow` đảm nhiệm việc giải mã hình ảnh. Hệ thống tận dụng sRGB textures của OpenGL để tự động chuyển đổi gamma sang không gian màu tuyến tính (linear color space), đảm bảo tính nhất quán về ánh sáng.

3.2 Kiến trúc Entity-Component-System (ECS)

Thay vì sử dụng Lập trình hướng đối tượng (OOP) truyền thống, hệ thống được thiết kế theo kiến trúc **ECS (Entity-Component-System)**. Kiến trúc này phân tách hoàn toàn dữ liệu ra khỏi logic xử lý, giúp tăng tính linh hoạt và tối ưu hóa việc lặp qua hàng loạt đối tượng trong đồ họa máy tính. Trung tâm của kiến trúc này là `Registry` quản lý toàn bộ vòng đời của thực thể và thành phần.

3.2.1 Thực thể (Entity) và Thành phần (Component)

Entity là những định danh nhẹ đại diện bằng số nguyên (`int`). Hệ thống hỗ trợ quan hệ cha-con (parent-child relationship), cho phép thực hiện dọn dẹp phân cấp (*hierarchical disposal*). Khi một Entity cha bị xóa, toàn bộ Entity con sẽ tự động được giải phóng để tránh rò rỉ bộ nhớ.

Component là các cấu trúc dữ liệu thuần túy (triển khai bằng `@dataclass`), định nghĩa thuộc tính của Entity:

- `Transform`: Lưu trữ vị trí, tỷ lệ và phép quay. Sử dụng Quaternion để tránh hiện tượng khóa trục (Gimbal lock) và đảm bảo nội suy mượt mà.
- `Visuals`: Liên kết Entity với Mesh và Material; quản lý chế độ vẽ (Fill/Wireframe) và thiết lập culling mặt sau (back-face culling).
- `Material`: Lưu trữ các tham số PBR bao gồm Albedo, Roughness, Metallic, Reflectance và Ambient Occlusion (AO), hỗ trợ cả giá trị hằng số và bản đồ texture.

- `Camera`: Định nghĩa thông số phối cảnh như FOV và các mặt phẳng cắt (clipping planes).
- `PointLight` và `DirectionalLight`: Định nghĩa nguồn sáng điểm (có suy giảm theo khoảng cách) và nguồn sáng hướng (song song, mô phỏng ánh sáng mặt trời).

Ngoài ra, hệ thống sử dụng các **Thành phần Singleton** để quản lý trạng thái toàn cục:

- `Disposal`: Danh sách Entity chờ giải phóng.
- `UiState`: Trạng thái điều hướng và Entity đang chọn.
- `CameraState`: Ma trận View/Projection và dữ liệu camera chính.
- `IconRenderState`: Cấu hình hiển thị biểu tượng trong viewport.
- `RenderState`: Quản lý handle FBO, texture, shader và dữ liệu bounding box.
- `AssetsState`: Quản lý thread pool, hàng đợi công việc và bộ nhớ đệm tài nguyên.
- `SpawnerState`: Yêu cầu khởi tạo Entity từ model đã nạp.
- `GizmoState`: Trạng thái tương tác và tính toán của hệ thống Gizmo.

Mã nguồn
Mã nguồn chứa các Component nằm ở `src/entities/components`.

3.2.2 Hệ thống xử lý (System)

System chứa logic nghiệp vụ, thực hiện các phép toán trên các Entity có Component tương ứng.

- **AssetSystem**: Là hệ thống bất đồng bộ quản lý vòng đời model và texture. Để tránh làm treo luồng render chính, hệ thống sử dụng một luồng worker chạy ngầm theo dõi `task_queue`. Khi yêu cầu tài nguyên, hệ thống trả về một proxy với trạng thái `AssetStatus.Loading`. Sau khi `trimesh` và `Pillow` xử lý xong, kết quả được đẩy vào `result_queue` để luồng chính thực hiện các thao tác nhạy cảm với OpenGL (tạo VAO, VBO, mipmap). Hệ thống có cơ chế caching theo đường dẫn file và xử lý lỗi chi tiết để tránh crash ứng dụng.
- **RenderSystem**: Thực thi pipeline render 3D sử dụng `tf2_ggx_shader`.
  - *Mô hình ánh sáng*: Áp dụng PBR nâng cao với thành phần *Hammon visibility* và *Hammon GGX diffuse* cho phản xạ gương và tán xạ bề mặt chính xác. **Chỉ có trên branch `master` cho Bài tập lớn 2. Bài tập lớn 1 ở branch `a1p1` chỉ có Phong, Gouraud và Flat.**
  - *Hậu xử lý*: Sử dụng *AgX tonemapping* để xử lý màu sắc HDR và hiệu ứng film grain.
  - *Sinh dữ liệu*: Thực hiện pass render phụ với `id_shader` ghi Entity ID vào framebuffer 32-bit unsigned integer (`R32UI`). Hệ thống đảm bảo các sub-mesh của một model phức tạp dùng chung ID cha để tạo mask phân đoạn nhất quán. Sau đó, sử dụng NumPy để kiếm các Entity không bị che trên mask, và ánh xạ Entity ID qua thể loại vật thể cho mask cuối. Cuối cùng, xuất đồng bộ ảnh RGB, depth map 16-bit, nhãn YOLO và segmentation mask.

- **UiSystem**: Quản lý giao diện qua `imgui-bundle`. Điểm nổi bật là cơ chế đồng bộ hai chiều giữa góc Euler và Quaternion: khi người dùng chỉnh Euler, hệ thống tính toán hai Quaternion tiềm năng và dùng *dot product* để chọn giá trị gần nhất, tránh hiện tượng "lật" hình. Đồng thời sử dụng hàm `minimize_euler` để giữ giá trị trong khoảng $[-180°, 180°]$. Ngoài ra, hệ thống còn phối hợp với `AssetSystem` để tạo mesh xem trước (Sphere, Torus, Cube) theo thời gian thực trước khi thêm vào scene.
- **CameraSystem**: Triển khai logic *Arcball Camera* (quay, tịnh tiến, thu phóng) và đồng bộ hóa vị trí camera với điểm tiêu điểm (focal point), cập nhật ma trận View/Projection vào `CameraState`.
- **GizmoSystem**: Cung cấp tay cầm tương tác 3D. Với phép tịnh tiến, hệ thống tính giao điểm của chuột với một mặt phẳng ảo song song với trục được chọn. Với phép quay, nó theo dõi delta góc của chuột so với tâm vật thể. Kích thước Gizmo được scale động theo khoảng cách đến camera để giữ nguyên kích cỡ trên màn hình.
- **SpawnerSystem**: Chịu trách nhiệm hiện thực hóa các asset đã nạp. Khi một model chuyển sang trạng thái `Ready`, hệ thống duyệt các node, tạo Entity tương ứng và gán các Component `Transform`, `Visuals`, `Material`, đồng thời chuyển đổi tọa độ cục bộ sang tọa độ thế giới.
- **IconRenderSystem**: Chiếu vị trí thế giới của các Entity "vô hình" (đèn, camera) lên màn hình 2D. Sử dụng background draw list của ImGui để vẽ các biểu tượng FontAwesome, giúp người dùng dễ dàng định vị và chọn đối tượng trong không gian.
- **DisposalSystem**: Quản lý việc xóa Entity an toàn. Hỗ trợ xóa đệ quy toàn bộ cây phân cấp trừ khi Entity được đánh dấu bằng `EntityFlags` đặc biệt.
- **TransformInheritanceSystem**: Cập nhật ma trận biến đổi dựa trên quan hệ cha-con, đảm bảo mọi biến đổi của vật thể cha được kế thừa chính xác bởi các vật thể con trong Scene Graph.

3.3 Các hệ thống chuyên biệt

Để phục vụ các bài tập lớn, hệ thống tích hợp thêm các module chuyên sâu:

- **SceneGeneratorSystem**: Tự động tạo môi trường đường phố phức tạp dựa trên cấu hình `SceneGeneratorState` (kích thước đường, mật độ nhà cửa). Hệ thống gán các nhãn phân loại (`Vehicle`, `Building`) để phục vụ RenderSystem sinh label chính xác.
- **SceneAnimatorSystem**: Mô phỏng giao thông dựa trên luật: tính toán gia tốc/phanh dựa trên khoảng cách để tránh va chạm, thực hiện chuyển làn mượt mà với hiệu ứng nghiêng thân xe, và áp dụng *world-wrapping* (vòng lặp thế giới) để duy trì luồng giao thông liên tục.
- **FunctionSurfaceSystem**: Phân tích các biểu thức toán học từ `SurfaceFunction` thông qua `numpy` để tạo lưới tam giác 3D. Hệ thống chỉ biên dịch lại mesh khi biểu thức hoặc tham số thay đổi.
- **GradientDescentSurfaceSystem**: Triển khai các thuật toán tối ưu hóa (SGD, Momentum) trên các hàm mất mát (ví dụ: Rosenbrock). Hệ thống cập nhật vị trí các Entity "Optimizer" theo gradient của bề mặt và ghi lại quỹ đạo để trực quan hóa.

Mã nguồn
Mã nguồn chứa các System nằm ở `src/entities/systems`.

4 Các tính năng đã thực hiện

Hệ thống hoàn thiện hầu hết các yêu cầu đề ra trong cả hai bài tập lớn, cụ thể như sau:

4.1 Bài tập lớn 1: Dựng hình và Trực quan hóa

- **Phần 1 - Vẽ các hình cơ bản:**
  - Hiện thực đầy đủ các hình 2D (tam giác, hình chữ nhật, đa giác đều, hình tròn, elip, hình thang, ngôi sao, mũi tên) và các khối 3D (lập phương, mặt cầu, trụ, nón, xuyến, lăng trụ).
  - Khối cầu được mô hình hóa qua 3 phương pháp: Chia nhỏ tứ diện, chiếu lưới vuông lên mặt cầu và sử dụng tọa độ kinh vĩ độ.
  - Hỗ trợ nhiều chế độ hiển thị: **Chỉ có trên branch `a1p1` trên kho GitHub.** Màu phẳng (Flat), nội suy đỉnh (Gouraud), chiếu sáng theo điểm ảnh (Phong), texture mapping, và khung dây (Wireframe).
  - Tích hợp hệ thống nhiều nguồn sáng và nhiều camera cho phép chuyển đổi và tương tác linh hoạt.
- **Phần 2 - Mô phỏng Gradient Descent:**
  - Trực quan hóa bề mặt (surface) và đường đồng mức (contour) của các hàm mất mát: Himmelblau, Rosenbrock, Booth và Quadratic.
  - Cho phép chạy song song nhiều bộ tối ưu hóa với các giải thuật: SGD, Momentum, Adam.
  - Người dùng có thể điều chỉnh trực tiếp các siêu tham số như learning rate, momentum và tốc độ mô phỏng ngay trong quá trình chạy, hay dùng gizmo để di chuyển optimizer.

4.2 Bài tập lớn 2: Bộ sinh ảnh tổng hợp

Hệ thống được thiết kế để tạo ra các tập dữ liệu huấn luyện cho bài toán xe tự hành với các thành phần:

- **Sinh dữ liệu đa tầng:** Xuất đồng thời ảnh RGB chất lượng cao, bản đồ độ sâu (depth map) và mặt nạ phân đoạn (segmentation mask) khớp hoàn toàn về độ phân giải.
- **Gán nhãn tự động:** Tự động tính toán và xuất nhãn hộp giới hạn (Bounding Box) theo định dạng YOLO.
- **Cấu hình cảnh:** Hệ thống tự động nạp các mô hình xe và môi trường từ file model kèm sẵn trong mã nguồn, cho phép ngẫu nhiên hóa vị trí, kèm theo đó thay đổi góc nhìn và nguồn sáng (các tính năng đã có từ Bài tập lớn 1) để tăng tính đa dạng cho bộ dữ liệu.

5 Hướng dẫn sử dụng

Ứng dụng cung cấp giao diện trực quan giúp người dùng dễ dàng làm quen và thao tác.

5.1 Điều khiển Camera và Quan sát

- **Xoay (Orbit):** Nhấn giữ và kéo **Chuột trái** để quay quanh điểm hội tụ.
- **Di chuyển (Pan):** Nhấn giữ **Chuột giữa** (hoặc con lăn) để di chuyển tịnh tiến camera.
- **Thu phóng (Zoom):** Sử dụng **Con lăn chuột** hoặc nhấn giữ **Chuột phải** để phóng to/thu nhỏ.
- **Tập trung (Focus):** Chọn một Entity trong danh sách và nhấn `Focus camera on this` để đặt tâm camera vào vật thể đó.

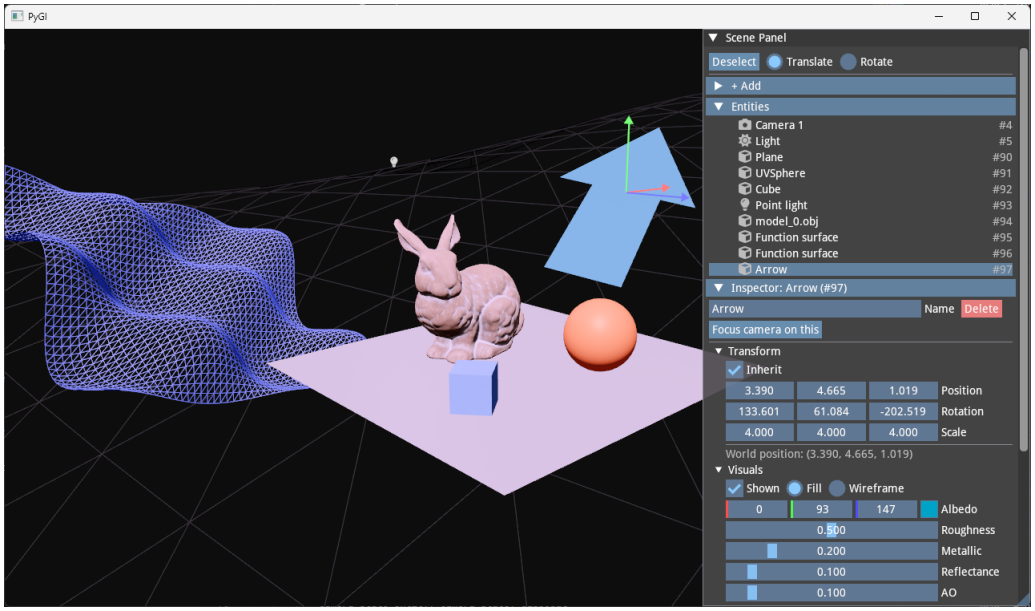
5.2 Quản lý và Tương tác Vật thể

- **Thêm vật thể:** Sử dụng menu `+ Add` trong Scene Panel để chọn khối cơ bản hoặc tải mô hình từ file. Một bản xem trước (preview) màu xanh sẽ xuất hiện trước khi người dùng xác nhận thêm vào cảnh.
- **Gizmo trực quan:** Khi chọn một vật thể, hệ thống Gizmo sẽ xuất hiện. Người dùng có thể chuyển đổi giữa chế độ `Translate` (di chuyển theo trục) và `Rotate` (xoay quanh trục) ngay trên viewport.
- **Inspector:** Cho phép chỉnh sửa chi tiết các thông số về Transform, thuộc tính vật liệu (Albedo, Roughness, Metallic, Reflectance, AO) và thay đổi texture (Albedo, Normal, Roughness, Metallic) cho từng vật thể.

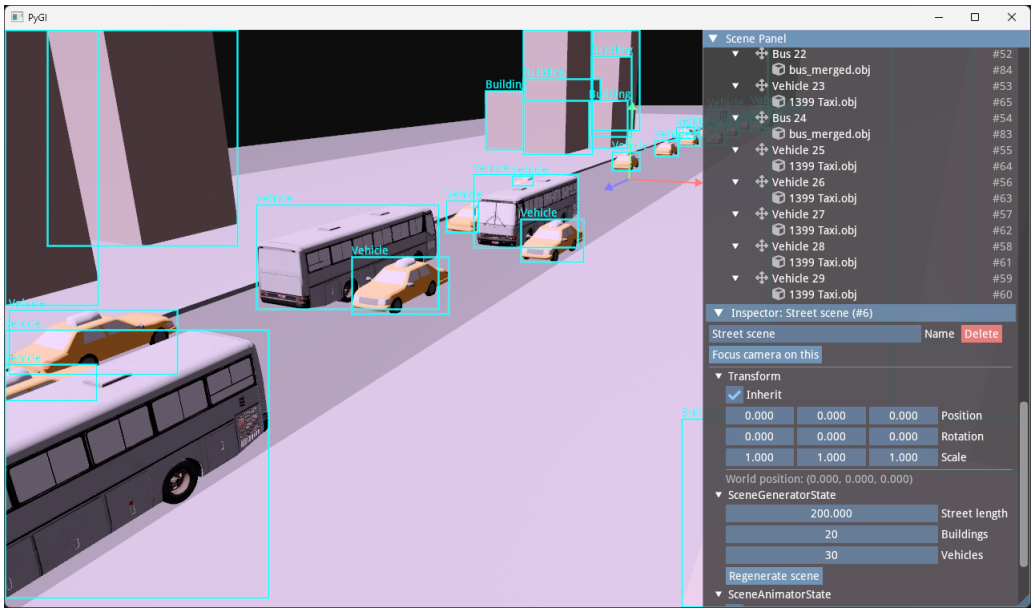
5.3 Công cụ chuyên dụng

- **Hàm toán học (Bài tập lớn 1):** Thêm `Function surface`, nhập biểu thức vào trường `Expression` (ví dụ: `np.sin(x) * np.cos(z)`). Nhấn `Compile` để cập nhật hình dạng. Có thể sử dụng những hàm NumPy như `np.sin`, `np.cos`, v.v..
- **Sinh dữ liệu (Bài tập lớn 2):** Khi khởi động ứng dụng, sẽ mặc định tạo một Entity **Street scene**. Khi chọn Entity này, có thể tinh chỉnh cảnh trong phần `Inspector`, như các thông số ngẫu nhiên hóa cảnh trong Component `SceneGeneratorState` và chạy cảnh trong `SceneAnimatorState`. Trong phần `Graphics`, nhấn `Capture this frame` hoặc `Start multi-frame capture` để hệ thống tự động lưu ảnh RGB, nhãn phân đoạn và bản đồ độ sâu vào thư mục `dataset/`.

6 Hình chụp màn hình

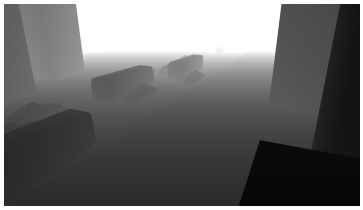


Hình 1: Dựng ảnh (Bài tập lớn 1)

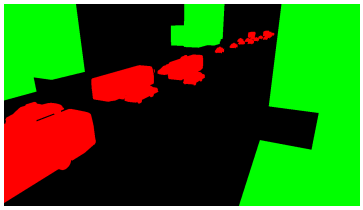


Hình 2: Dựng ảnh (Bài tập lớn 2)

Hình 3: Ảnh RGB



Hình 4: Ảnh Depth



Hình 5: Ảnh Segmentation

7 Kho GitHub & hướng dẫn cài đặt

Toàn bộ mã nguồn được lưu trữ tại:

https://github.com/nghiaho310pf/PyGl

7.1 Cài đặt

7.1.1 Yêu cầu hệ thống

- **Python 3.10** trở lên.
- Hệ thống hỗ trợ **OpenGL 4.5** trở lên (OpenGL 4.1 đối với macOS).

7.1.2 Các bước cài đặt & chạy

1. **Clone kho GitHub**:

```
git clone https://github.com/nghiaho310pf/PyGl
cd PyGl
```

2. **Tạo môi trường ảo (virtual environment)**:

```
python -m venv .venv
source .venv/bin/activate  # Trên Windows: .venv\Scripts\activate
```

3. **Cài đặt các thư viện phụ thuộc**:

```
pip install -r requirements.txt
```

4. **Chạy ứng dụng:**

```
python src/main.py
```

7.1.3 Khắc phục sự cố

- **Lỗi OpenGL:** Nếu gặp lỗi liên quan đến phiên bản OpenGL, cần đảm bảo driver đã được cập nhật. Có thể kiểm tra phiên bản OpenGL bằng các công cụ như `glxinfo` (Linux) hoặc `GPU-Z` (Windows).
- **Lỗi import:** Đảm bảo đã thực hiện thành công các bước trên.

- **Tải mô hình:** Ứng dụng sử dụng thư viện `trimesh` để tải mô hình. Nếu một số định dạng mô hình (ví dụ: `.glb`) không tải được, đảm bảo đã cài đặt các thư viện phụ thuộc bổ sung của nó (ví dụ: chạy `pip install trimesh[all]`).